MATLAB File Handling and Engineering Data Processing

A Line-by-Line Walkthrough of the COE158 Activity Sheet

Contents

About This Worksheet

Every program so far has worked with data you typed in by hand, using `input`. Real engineering work rarely starts that way — data usually already exists somewhere: a sensor log, a spreadsheet from a lab session, a photo of a circuit board. This worksheet teaches you how to move data **in and out** of MATLAB using the file formats you'll actually encounter: plain text files, CSV files, Excel files, and images. The same two strategies come up again and again for every format — **bulk** (grab everything in one go) and **line-by-line / range** (process it piece by piece, with more control) — so once you understand that pattern, the rest is mostly learning which function name goes with which file type.

---

1. Bulk Reading a Text File — `readmatrix`

```
data = readmatrix('engineering_data.txt');
Voltage = data(:,1);
Current = data(:,2);
disp(data)
```

```
Power = Voltage .* Current;
disp(Power)
```

`readmatrix` opens the named file, reads **all** of its numeric contents, and stores it as a single MATLAB matrix in one line — MATLAB automatically figures out how many rows and columns the file has. Once `data` exists, `data(:,1)` means "every row, column 1" (see the earlier notes on matrix indexing) — this is how the two columns of the file are split apart into separate, meaningfully-named variables, `Voltage` and `Current`.

`Power = Voltage .* Current;` uses the **element-wise** multiplication operator `.*` rather than plain `*`. This matters because `Voltage` and `Current` are both column vectors of the same size — `.*` multiplies each voltage by its *own* corresponding current (row 1 with row 1, row 2 with row 2, and so on), which is what you want physically. Plain `*` would instead attempt full matrix multiplication and likely produce an error, since two column vectors of the same size aren't compatible under standard matrix-multiplication rules.

Why it's used here: `readmatrix` is the fastest, simplest way to get a whole file of clean numeric data into MATLAB, and it's the method you'll reach for by default whenever a file is small enough, and regularly formatted enough, to load all at once.

---

2. Line-by-Line Reading a Text File — `fopen`, `fgetl`, `sscanf`, `fclose`

```
fileID = fopen('engineering_data.txt','r');
line = fgetl(fileID);
while ischar(line)
    data = sscanf(line,'%f');
    Voltage = data(1);
    Current = data(2);
    Power = Voltage * Current;
    fprintf('Voltage = %.2f V, Current = %.2f A, Power = %.2f W\n', ...
        Voltage, Current, Power);
    line = fgetl(fileID);
end
fclose(fileID);
```

Reading line-by-line takes more setup than `readmatrix`, but gives you far more control — useful for files too large to load all at once, or where you need to do something different with each individual row.

- **`fopen('engineering_data.txt','r')`** opens the file for **reading** (`'r'`) and hands you back a `fileID` — think of this as a "handle" or ticket that every subsequent command uses to refer to *this specific open file*.
- **`fgetl(fileID)`** reads exactly **one line** of text from the file (without the newline character at the end) and moves an internal "cursor" forward, so the *next* call to `fgetl` reads the line after it.
- **`while ischar(line)`** is the loop's stopping condition, and it's worth understanding exactly why it works: as long as `fgetl` successfully reads a real line, it returns

that line as text (a char array). But once you've read past the very last line, `fgetl` instead returns the number `-1` — and `-1` is **not** text, so `ischar(-1)` is false, and the loop stops automatically. This is a common MATLAB pattern: "keep looping while what I just read is still text."
- **`sscanf(line,'%f')`** takes the one line of text you just read (e.g. `"10.5 2.3"`) and converts it into actual numbers, according to the format `'%f'` (meaning "read floating-point numbers"). It returns them as a small vector, so `data(1)` and `data(2)` pull out the first and second numbers on that line.
- **`fclose(fileID)`** closes the file once you're done. This isn't optional housekeeping — an unclosed file can stay "locked" by MATLAB, causing errors if you try to open or edit it again later in the same session.

Why it's used here: this shows you the manual version of what `readmatrix` does automatically, one line at a time, which becomes essential once a file is either too large to load in bulk, or needs custom processing applied to each row individually (as you'll see with CSV headers next).

---

3. The Key Difference: Bulk vs Line-by-Line

Bulk Reading	Line-by-Line Reading
Reads many/all data at once	Reads one line at a time
Simple and convenient	More control over processing
Suitable for structured numerical data	Useful for large or mixed-format files
`readmatrix()`	`fopen()` + `fgetl()`

The trade-off is speed/simplicity versus control. Default to bulk reading unless you have a specific reason not to — a file too big to fit comfortably in memory, a file where different lines need different treatment, or a file with a header row you need to explicitly skip (see the CSV section below).

---

4. Bulk Writing a Text File — `writematrix`

```
Voltage = [10.5; 12.0; 15.5; 18.0; 20.0];
Current = [2.3; 2.8; 3.4; 4.1; 4.8];
data = [Voltage Current];
writematrix(data,'engineering_data.txt');
```

`[Voltage Current]` combines two column vectors **side-by-side** into a single matrix — a space (or comma) between them inside square brackets means "place these next to each other as new columns," which only works because `Voltage` and `Current` have the same number of rows. `writematrix` then takes that matrix and writes it straight to a file, creating the file if it doesn't already exist, or overwriting it if it does.

Why it's used here: it's the direct mirror image of `readmatrix` — the simplest possible way to export a finished matrix of results, with no formatting decisions required from you.

---

5. Line-by-Line Writing — `fopen('w')` and `fprintf` to a File

```
fileID = fopen('engineering_data.txt','w');
for i = 1:length(Voltage)
    fprintf(fileID,'%.2f %.2f\n', ...
        Voltage(i), Current(i));
end
fclose(fileID);
```

This reuses `fopen`, but with `'w'` instead of `'r'` — meaning "open this file for **writing**" (and create it if it doesn't exist yet; be careful, as `'w'` also erases any existing content in that file the moment you open it). Notice `fprintf` here takes `fileID` as its **first** argument — this is what tells MATLAB to send the formatted text into the **file** rather than printing it to the Command Window, which is the only difference from the `fprintf` you've used before.

The `for` loop runs once per row (`length(Voltage)` gives the number of entries), writing one voltage/current pair per line. The `...` at the end of the `fprintf` line is a **line-continuation marker** — it tells MATLAB "this statement isn't finished yet, keep reading on the next line," purely so the code doesn't run off the edge of the page/screen. It has no effect on what the program does.

Why it's used here: line-by-line writing is essential when your data doesn't already exist as one neat matrix — for example, values calculated one at a time inside a loop, where writing each result the moment it's ready is more natural than collecting everything first.

---

6. CSV Files — Same Ideas, With a Header Row

```
data = readmatrix('engineering_data.csv');   % bulk reading — identical to .txt

fileID = fopen('engineering_data.csv','r');
header = fgetl(fileID);                       % read and discard the header row
line = fgetl(fileID);
while ischar(line)
    data = sscanf(line,'%f,%f');              % note the comma in the format
    ...
```

A CSV (**C**omma-**S**eparated **V**alues) file is really just a text file where commas mark the column boundaries, which is why `readmatrix` handles both formats identically — you don't need a different function at all for bulk reading.

Line-by-line reading needs one small but important addition: **`header = fgetl(fileID);`** reads the very first line (`Voltage,Current`) and stores it in a variable that's simply never used again. This isn't wasted code — it's a deliberate way to "use up" and discard the header row, so the `while` loop that follows only ever sees the actual numeric rows. Skip this line, and `sscanf` would try (and fail) to convert the word "Voltage" into a number.

The format string also changes slightly, from `'%f'` to `'%f,%f'` — the comma inside the format string tells `sscanf` to expect a comma **between** the two numbers on the line, matching the CSV's structure.

Writing a CSV with a header follows the same idea in reverse:

```
fprintf(fileID,'Voltage,Current\n');   % write the header first
for i = 1:length(Voltage)
    fprintf(fileID,'%.2f,%.2f\n', Voltage(i), Current(i));
end
```

One extra `fprintf` call, before the loop, writes the column headings as the very first line of the file — everything after that is identical to writing a plain text file, just with commas instead of spaces between values.

---

7. Excel Files — Bulk Reading, Ranges, and `writecell`

```
data = readmatrix('engineering_data.xlsx');           % whole worksheet
data = readmatrix('engineering_data.xlsx','Range','A2:B6');  % just one range
```

`readmatrix` also works directly on `.xlsx` files — MATLAB detects the file type from its extension and handles the Excel-specific reading behind the scenes. The optional `'Range','A2:B6'` argument tells it to import **only** the cells inside that rectangle, using standard spreadsheet cell references (A2 = column A, row 2). This is specifically useful when a worksheet has headings, notes, or other cells you don't want mixed into your numeric data.

Writing to Excel introduces one new function, `writecell`:

```
writecell({'Voltage','Current'}, 'engineering_data.xlsx','Range','A1');
writematrix(data, 'engineering_data.xlsx','Range','A2');
```

`writematrix` only knows how to write **numbers** — it can't write the text labels "Voltage" and "Current" directly. `writecell` is the version for writing mixed or text data instead, and `{'Voltage','Current'}` with **curly braces** creates a **cell array** — a container that (unlike a normal matrix) is allowed to hold different types of data, here two pieces of text. Writing the headings with `writecell` to Range `A1` and the numeric data with `writematrix` to Range `A2` (starting one row below) is how the two pieces are combined into one properly-labelled spreadsheet.

---

8. Reading Images — `imread`, `imshow`, `size`, `whos`, `imfinfo`

```
img = imread('circuit_board.jpg');
imshow(img)
size(img)
whos img
```

`imread` loads an image file into MATLAB as a matrix of pixel values, stored in the variable `img`. `imshow` displays it in a Figure window, exactly like `plot` would display a graph. `size(img)` typically returns **three** numbers for a colour photo — e.g. `480 640 3` — meaning 480 pixels tall, 640 pixels wide, and 3 **colour channels** (Red, Green, Blue values stored separately for every pixel). `whos img` gives you a fuller summary of the same variable: its size, its data type, and how much memory it's using.

```
info = imfinfo('circuit_board.jpg');
disp(info.Width)
disp(info.Height)
disp(info.Format)
```

`imfinfo` is different from `imread` in an important way: it reads the image file's **metadata** (width, height, format, file size, and more) **without** loading the actual pixel data into memory. This matters for large images or batches of files where you just need to check dimensions or format quickly, without the cost of loading every pixel. The result, `info`, is a **structure** — a variable that groups several related pieces of information together under named fields, which is why you access them with a dot: `info.Width`, `info.Height`, `info.Format`.

Why it's used here: `imread` is for when you actually need to *look at* or *process* the image; `imfinfo` is for when you only need to *know something about* it.

---

9. Writing Images — `imwrite`, and Processing Before Saving

```
img = uint8(randi([0 255],200,300,3));
imshow(img)
imwrite(img,'generated_image.png');
```

`randi([0 255],200,300,3)` generates random whole numbers between 0 and 255, arranged into a 200×300×3 array — matching the height/width/colour-channel structure `size(img)` showed you earlier, so this is really just a randomly-generated "photo." `uint8` converts those numbers into **8-bit unsigned integers**, the specific data type image files expect pixel values to be stored as (this is one of the few places in this whole worksheet series where the numeric *type*, not just the value, genuinely matters). `imwrite` then saves that pixel data to an actual image file, with the format (`.png` here) determined by the file extension you give it.

Processing an image before saving follows the same read → modify → write pattern you've seen with matrices and text files:

```
img = imread('circuit_board.jpg');
grayImage = rgb2gray(img);
```

```
imshow(grayImage)
imwrite(grayImage,'circuit_board_gray.png');
```

`rgb2gray` converts a 3-channel colour image into a single-channel greyscale one, by combining the Red, Green, and Blue values of each pixel into one brightness value. The result, `grayImage`, is then displayed and saved exactly like any other image — `imwrite` doesn't care whether the image came from a camera or was calculated by another MATLAB function first.

Summary — Quick Reference

Task	Bulk / Whole-File Function	Line-by-Line / Range Function
Read a text file	`readmatrix('file.txt')`	`fopen` + `fgetl` + `sscanf`
Write a text file	`writematrix(data,'file.txt')`	`fopen('...','w')` + `fprintf` + `fclose`
Read a CSV file	`readmatrix('file.csv')`	`fopen` + `fgetl` (skip header) + `sscanf`
Write a CSV file	`writematrix(data,'file.csv')`	`fprintf` header line, then loop + `fprintf`
Read an Excel file	`readmatrix('file.xlsx')`	`readmatrix(...,'Range','A2:B6')`
Write an Excel file	`writematrix(data,'file.xlsx')`	`writecell(...,'Range','A1')` + `writematrix(...,'Range','A2')`
Read an image	`imread('file.jpg')` + `imshow`	`imfinfo('file.jpg')` (metadata only)
Write an image	`imwrite(img,'file.png')`	— (same function; process the image first, e.g. `rgb2gray`)

Key functions used throughout:

Function	Purpose
`readmatrix`	Load an entire text/CSV/Excel file of numbers at once
`writematrix`	Save an entire matrix of numbers to a text/CSV/Excel file
`writecell`	Save text (or mixed) data, e.g. column headings, using {}
`fopen` / `fclose`	Open a file for reading (`'r'`) or writing (`'w'`); always close it after
`fgetl`	Read one line of text from an open file; returns `-1` at end of file
`ischar`	Checks whether a value is text — used to detect end-of-file with `fgetl`
`sscanf`	Convert a line of text into numbers, using a format like `'%f'`
`.*`	Element-wise multiplication — multiplies matching entries of two vectors/matrices
`imread` / `imshow`	Load an image into MATLAB / display it in a Figure window
`imfinfo`	Read an image's metadata (width, height, format) without loading its pixels
`imwrite`	Save an image (generated or processed) to a file
`rgb2gray`	Convert a colour image to greyscale

Big ideas to take away:

1. Bulk functions (`readmatrix`/`writematrix`) are simpler and are your default choice; line-by-line (`fopen`/`fgetl`/`fprintf`) gives you control when you need it.
2. Always pair `fopen` with a matching `fclose` — an open file left dangling can cause problems later in the same session.
3. `readmatrix` treats `.txt`, `.csv`, and `.xlsx` almost identically — the file extension does the work of telling MATLAB how to read it.
4. A CSV or Excel header row must be explicitly skipped (`fgetl` for the header) or excluded (`'Range','A2:B6'`) — MATLAB will otherwise try, and fail, to read text as numbers.
5. `imread` loads pixel data for display/processing; `imfinfo` reads only the metadata, which is faster when that's all you need.

Assignments and Practice Examples

Assignment 1 — Bulk Round-Trip

Create a text file `rainfall_data.txt` containing three columns — Day, Rainfall (mm), Temperature (°C) — for 6 made-up days. Read it in bulk with `readmatrix`, calculate the average rainfall using MATLAB's built-in `mean()` function, then write a **new** file called `rainfall_summary.txt` containing just the day with the highest rainfall and its value.

Assignment 2 — Line-by-Line with a Twist

Using the `temperature_data.txt` file from Practice Exercise 2 (or recreate it), read it **line-by-line** and, instead of just displaying the pressure difference, count how many measurements had a pressure **below** 101.3 kPa. Display that count once the loop finishes.

Assignment 3 — CSV Header Bug Hunt

Take the CSV line-by-line reading code from this worksheet and **remove** the `header = fgetl(fileID);` line. Run it and observe exactly what error MATLAB gives you. In one or two sentences, explain in your own words why removing that line causes the problem, referencing what `sscanf` is trying to do with the word "Voltage."

Assignment 4 — Excel Range Practice

Create an Excel file with three columns (Time, Voltage, Current) and **10** rows, with a heading row at the top. Using `readmatrix` with a `'Range'` argument, read only rows 3 through 7 (skipping the heading and the first two data rows), and display what you imported.

Assignment 5 — Image Pipeline

Find any `.jpg` image, save it in your Current Folder, and write a script that: reads it with `imread`, displays its width and height using `imfinfo`, converts it to greyscale with `rgb2gray`, displays both the original and greyscale versions (hint: use `figure;` before the second `imshow` so you get two separate windows), and saves the greyscale version with a new filename.

Assignment 6 — Reflection

1. Why does `writematrix` need the data combined into a single matrix first (`data = [Voltage Current]`), rather than being given `Voltage` and `Current` separately?
2. In your own words, explain why `ischar(line)` is a reliable way to detect the end of a file when using `fgetl`.
3. Describe a real engineering scenario where you would prefer `imfinfo` over `imread`, and explain why.